

14. Repaso de regresión + Recetas

Ciencia de datos para la toma de decisiones I

Jorge Juvenal
Campos
Ferreira

✉ juvenal.campos@tec.mx

Repaso clase previa

La clase pasada generamos un modelo de regresión lineal simple para predecir el precio de las casas.

Paso 1. Cargamos *tidymodels* y cargamos los datos

```
options(scipen = 999)

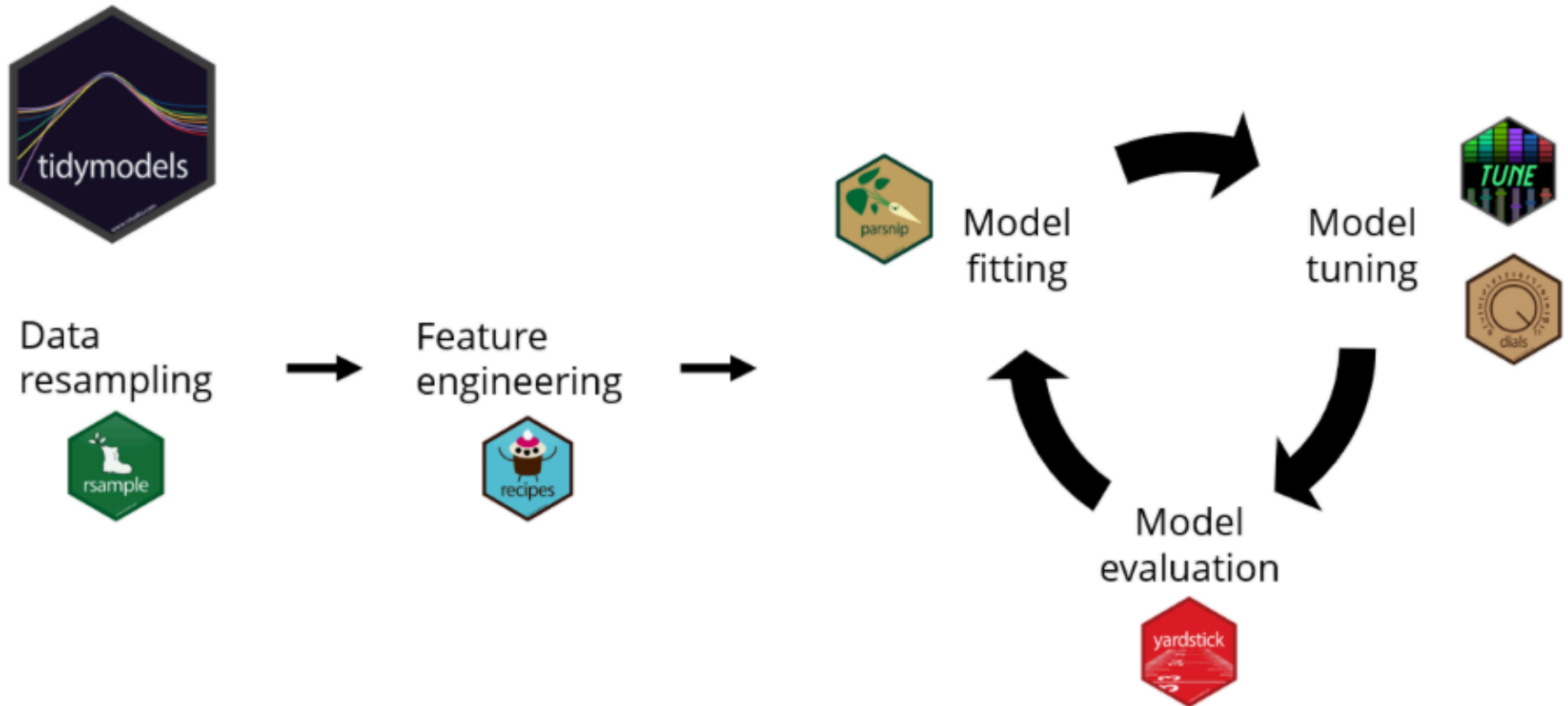
# Librerías ----
library(tidyverse)
library(tidymodels)

# 1. Cargue los datos del objeto home_sales.rds
home_prices <- readRDS("home_sales.rds")
```

Repaso clase previa

Paso 1. Cargamos *tidymodels* y cargamos los datos

Collection of machine learning packages



Repaso clase previa

Paso 2.Dividimos la base total en base de entrenamiento y base de prueba

```
# Utilizando initial_split(), divida la muestra en base de  
entrenamiento y base de prueba.
```

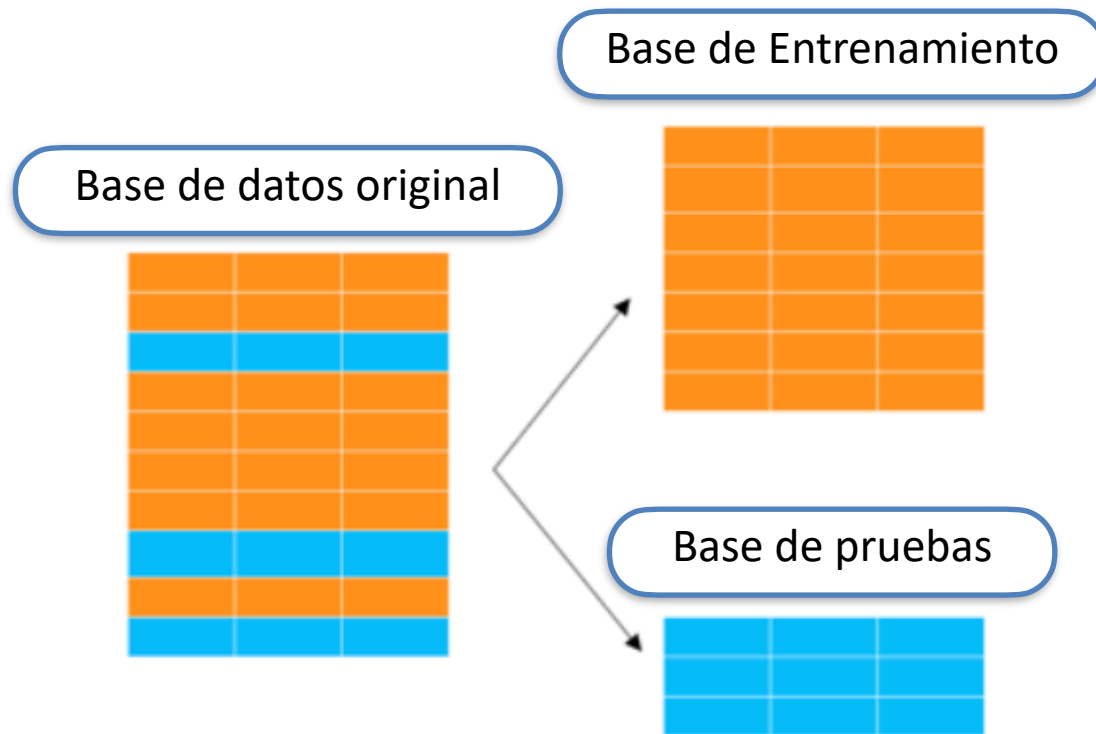
```
home_split <- initial_split(data = home_prices,  
                             prop = 0.7,  
                             strata = selling_price)
```

```
# Crear base de entrenamiento  
home_training <- home_split %>% training()
```

```
# Crear base de prueba  
home_test <- home_split %>% testing()
```

Repaso clase previa

Paso 2. Dividimos la base total en base de entrenamiento y base de prueba



Data resampling o remuestreo de datos

Repaso clase previa

Paso 3. Hicimos estadística descriptiva de ambas bases.

3. La variable y va a ser la variable `selling_price`. Obtenga, para la base de `entrenamiento` y la base de `prueba`, las estadísticas descriptivas de esta variable (`min`, `max`, `desviación estándar`, `media`). ¿Son diferentes?

```
set.seed(19910708)
```

```
home_training %>%
```

```
  summarise(media_precio = mean(selling_price, na.rm = T),
            min_precio = min(selling_price, na.rm = T),
            max_precio = max(selling_price, na.rm = T),
            sd_precio = sd(selling_price, na.rm = T))
```

```
home_test %>%
```

```
  summarise(media_precio = mean(selling_price, na.rm = T),
            min_precio = min(selling_price, na.rm = T),
            max_precio = max(selling_price, na.rm = T),
            sd_precio = sd(selling_price, na.rm = T))
```

ED Base de entrenamiento:

```
A tibble: 1 × 4
  media_precio min_precio max_precio sd_precio
    <dbl>         <dbl>         <dbl>         <dbl>
1   479497.         350000         650000         81614.
```

ED Base de prueba:

```
# A tibble: 1 × 4
  media_precio min_precio max_precio sd_precio
    <dbl>         <dbl>         <dbl>         <dbl>
1   478129.         350000         650000         79571.
```

= Muy parecidos

Repaso clase previa

Paso 4. Definimos el modelo

4. Defina un modelo de regresión lineal, utilizando el engine “lm” y el modo “regression”.

```
linear_model <- linear_reg() %>%  
  set_engine("lm") %>%  
  set_mode("regression")
```

El modelo es la primera pieza (de Lego) que vamos a armar en este **flujo de trabajo** de armar modelos de Machine Learning.

Repaso clase previa

Paso 4. Definimos el modelo

El problema de la regresión lineal consiste en encontrar los coeficientes beta (β) tales que minimicen la siguiente función de costo del error cuadrático:

$$\min_{\beta_0, \beta_1} S(\beta_0, \beta_1) = \sum_{i=1}^n (y_i - \beta_0 - \beta_1 x_i)^2.$$

Llegando a la siguiente solución:

$$\hat{\beta}_0 = \bar{y} - \hat{\beta}_1 \bar{x}.$$

$$\hat{\beta}_1 = \frac{\sum_{i=1}^n (x_i - \bar{x})(y_i - \bar{y})}{\sum_{i=1}^n (x_i - \bar{x})^2} = \frac{\widehat{\text{Cov}}(x, y)}{\widehat{\text{Var}}(x)}.$$

**Caso de una sola variable dependiente.*

Repaso clase previa

Paso 5. Ajustamos la fórmula

Tomando como variables X a las variables `home_age` y `sqft_living`, ajuste un modelo de regresión sobre la base de entrenamiento. ¿Cuál es la fórmula de la recta de regresión correspondiente?

```
lm_fit <- linear_model %>%  
  fit(formula = selling_price ~ home_age + sqft_living,  
      data = home_training)  
lm_fit
```

Ya con el modelo, ajustamos la fórmula de regresión que queremos probar.

Fórmula:

variable_dependiente ~ variable_independiente_1 + variable_independiente_2 + ...
+ variable_independiente_n

Repaso clase previa

Paso 6. Predicción de valores

6. Utilice este modelo con la base de prueba utilizando la función predict. “Péguele” la columna de valores predichos a la base de prueba.

```
home_predictions <- predict(lm_fit, new_data = home_test)
```

```
home_test_results <- home_test %>%  
  select(selling_price, home_age, sqft_living) %>%  
  bind_cols(home_predictions)
```

Ya con el modelo creado con la **base de entrenamiento**, lo probamos generando las predicciones con los valores nuevos (la **base de prueba**). Esto nos **predecirá** los valores de los precios que deberían tener las casas de la **base de prueba**, dadas sus características (**X**).

Repaso clase previa

Paso 7. Probamos la exactitud del modelo

```
# 7. Con estos datos en una sola tabla (reales y predichos por  
el modelo) obtenga el RMSE (Raíz del error cuadrático medio) y  
la  $R^2$ 
```

```
home_test_results %>%  
  rmse(truth = selling_price, estimate = .pred)
```

```
mean(home_test_results$selling_price)
```

```
home_test_results %>%  
  rsq(truth = selling_price, estimate = .pred)
```

Dado que tenemos valores **reales** y valores **predichos** por el modelo, verificamos que tan bien hizo el modelo para acercarse a estos valores.

Repaso clase previa

Paso 7. Probamos la exactitud del modelo

Dos métricas para evaluar la regresión:

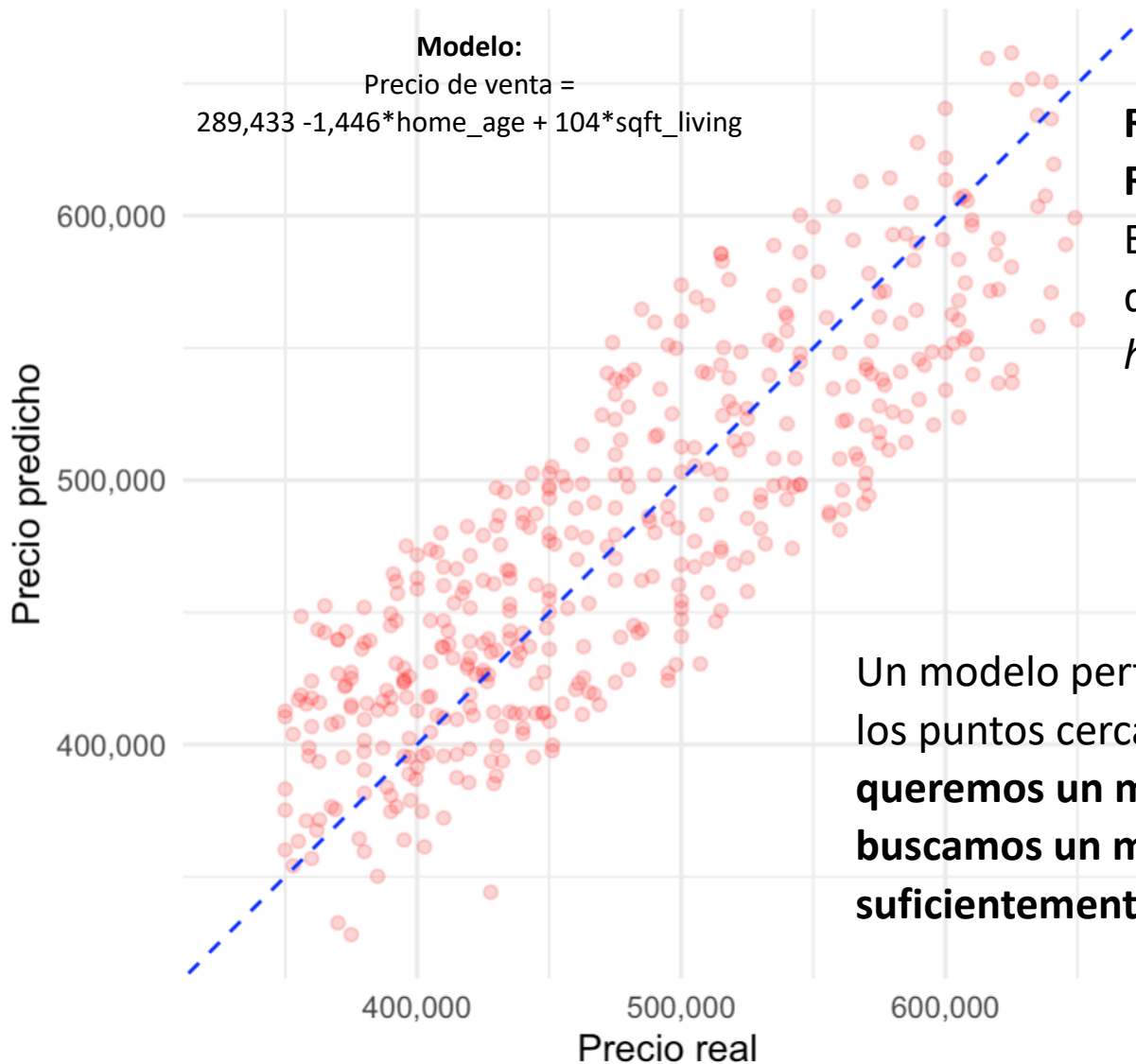
R^2

Es el valor que mide que tan bien se ajustan los datos a una línea de regresión.

RMSQ: Raíz del Error Cuadrado medio.

Es el valor que mide que tanto se alejan estos valores predichos de la realidad. Están en la misma unidad de medida de los valores reales.

Precios real vs predicho



Primer intento:

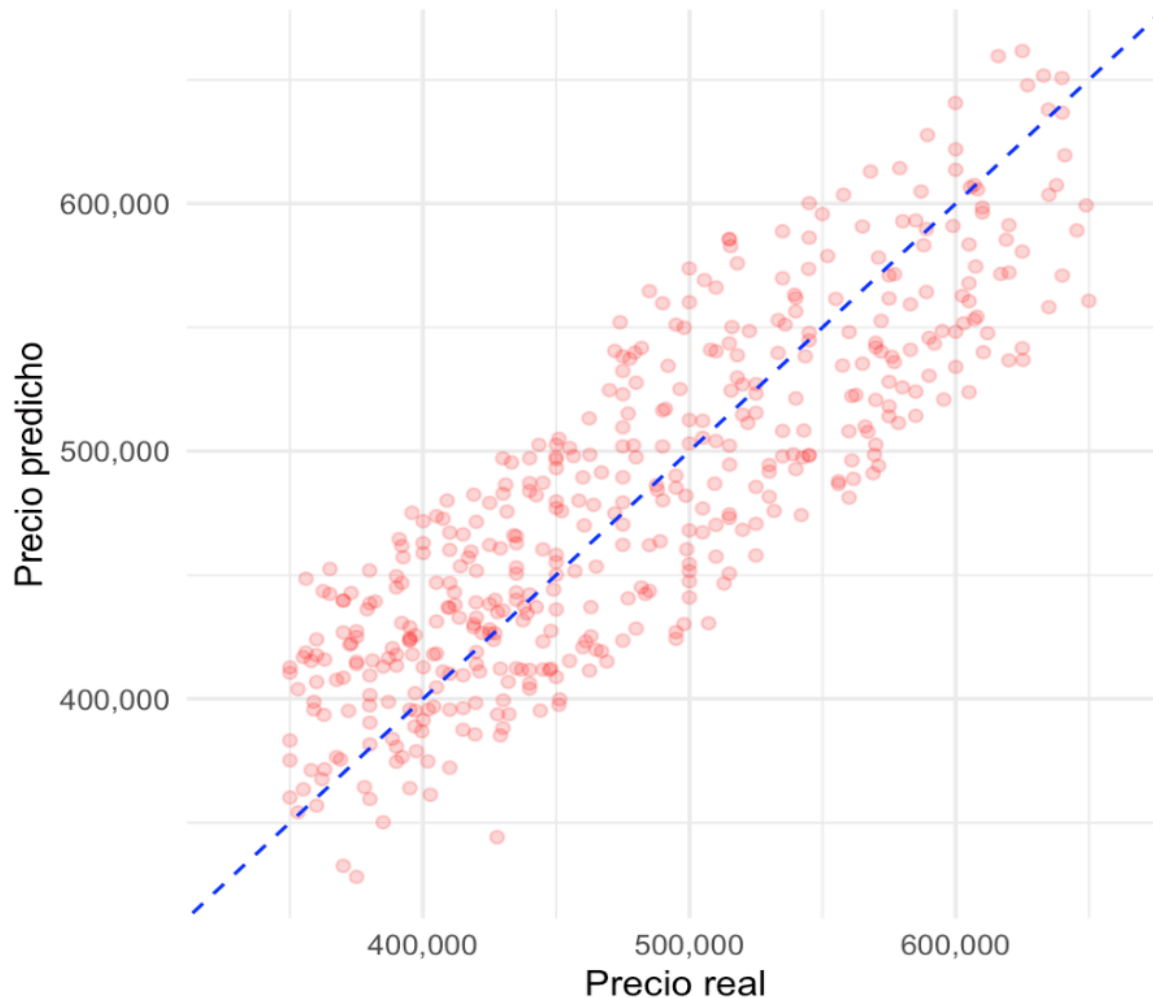
RMSQ: \$47,777

R²: 0.639

El primer intento solo utilizó
dos predictores (X):
home_age y *sqft_living*

Un modelo perfecto debería tener todos
los puntos cerca de la línea azul. **No
queremos un modelo perfecto, solo
buscamos un modelo que sea lo
suficientemente bueno.**

Precios real vs predicho



Segundo intento:

RMSQ: \$41,938

R²: 0.722

El segundo intento utilizó **todas** las variables explicativas (X) disponibles en la base. Esto implicó una mejoría en el **RMSQ** (que bajó) y un aumento en el **R²** (que subió).

Modelo 2:

Precio de venta =

$$256,713.67 - 993.86 * \text{home_age} + 44724.73 * \text{bedrooms} + 11352.51 * \text{bathrooms} + 143.87 * \text{sqft_living} + 0.12 * \text{sqft_lot} + -3.8 * \text{sqft_basement} + 32683 * \text{floors}$$

Repaso clase previa

Paso 8. Definir cuál es el mejor modelo

Primer intento:

RMSQ: \$47,777

R²: 0.639

El primer intento solo utilizó dos predictores (**X**): *home_age* y *sqft_living*

Segundo intento:

RMSQ: \$41,938

R²: 0.722

El segundo intento utilizó **todas** las variables explicativas (**X**) disponibles en la base. Esto implicó una mejoría en el **RMSQ** (que bajó) y un aumento en el **R²** (que subió).

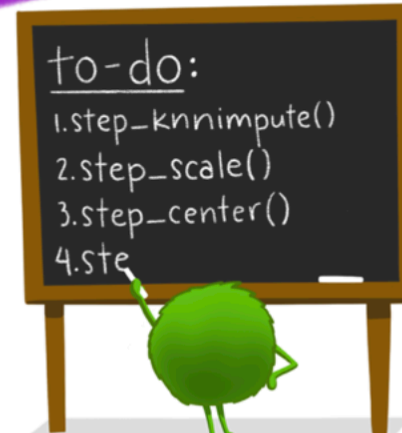
El segundo modelo es **mejor**, ya que tiene una Raíz del Error cuadrático medio menor, y un ajuste mayor.

VARIABLES
PANTRY



1. SPECIFY VARIABLES
recipe(y~a+b+..., data=pantry)

recipes:



2. DEFINE
PRE-PROCESSING
STEPS (step_*)

STREAMLINED DATA PRE-PROCESSING FOR
STATISTICAL + MACHINE LEARNING MODELS



3. PROVIDE
DATASET(S) FOR
RECIPE STEPS
prep()



4. APPLY
PRE-PROCESSING!
bake()

Artwork by @allison_horst

Recetas y Feature Engineering

★ Feature engineering

Es el proceso de transformar los datos crudos en **variables predictoras (X)** (*features*) que representen mejor el problema subyacente para un modelo predictivo, con el objetivo de mejorar su desempeño.

Principales acciones:

1. Crear nuevas

variables. Ej: extraer el día de la semana de una fecha, calcular una interacción, generar un rezago (lag).

2. Transformar variables.

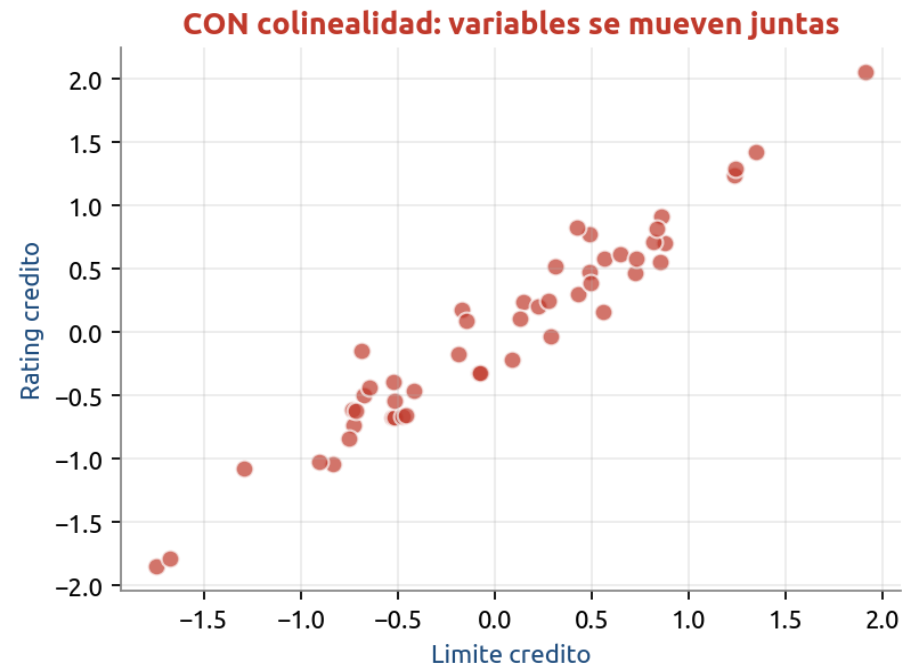
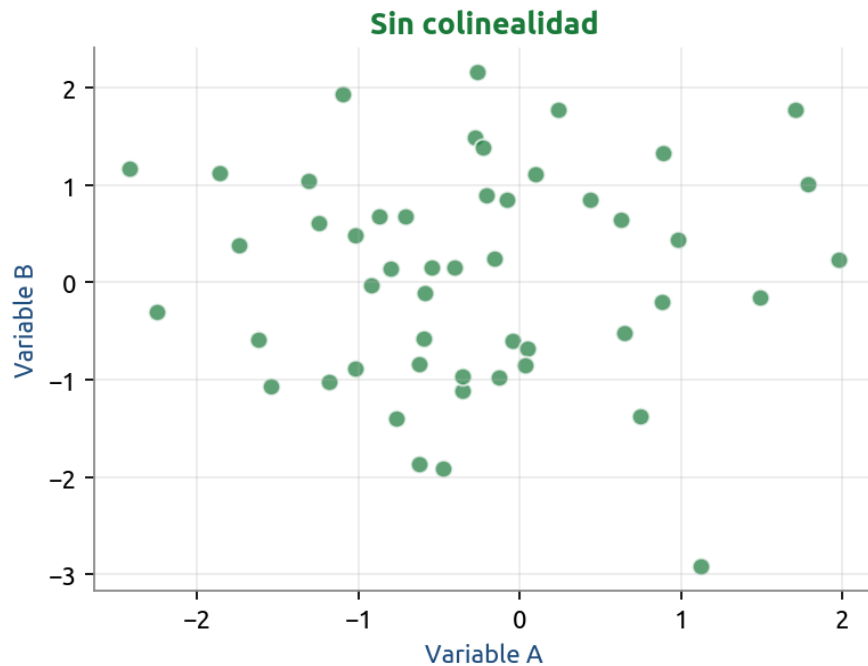
Para que cumplan los supuestos del modelo o capturen mejor la relación con la variable respuesta. (Logaritmos, normalización)

3. Codificar variables.

(Dummies, target encoding, hashing, embeddings)

Problema típico: colinealidad

Cuando dos predictores cuentan básicamente la misma historia.



Cómo detectarla

- Matriz de correlación de los predictores
- VIF (Variance Inflation Factor) > 5 o 10

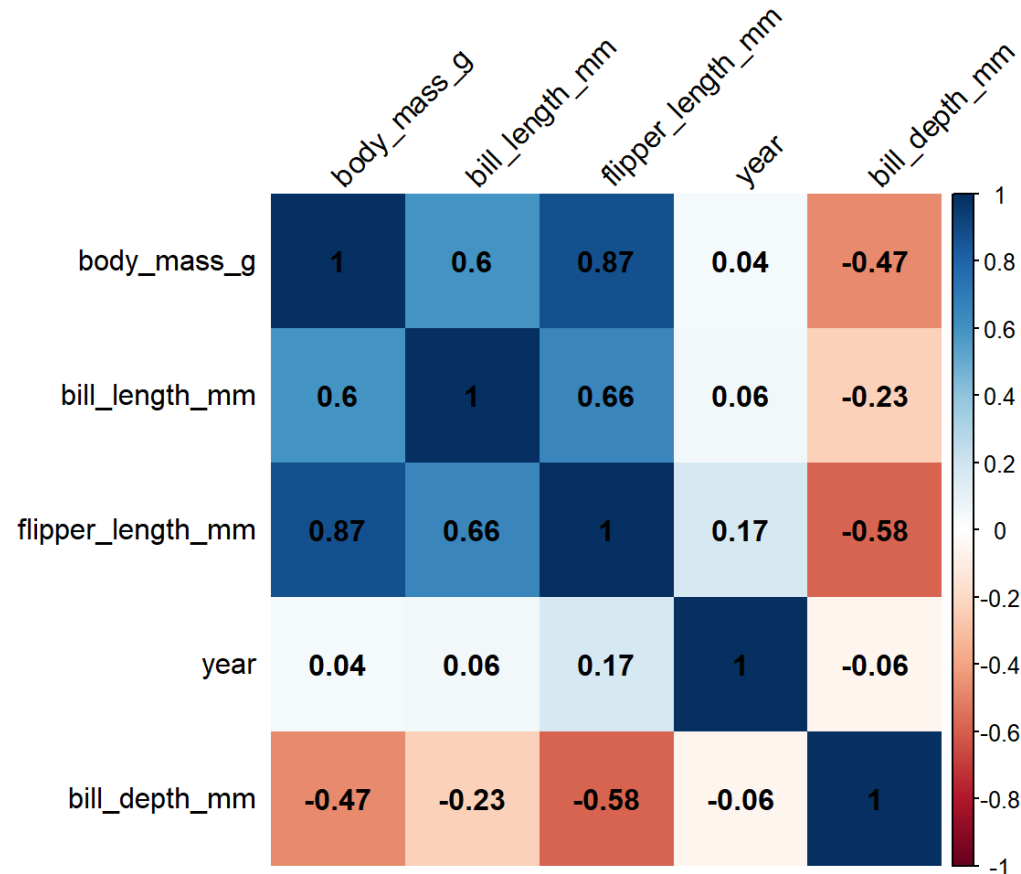
Cómo solucionarla

- Eliminar una de las variables
- Combinar las variables en una sola
- Usar regularización (Ridge, Lasso)

Matríz de correlación

Visualización que nos permite ver la correlación que hay entre distintas variables de los datos.

Se genera con la funcion `corrplot()` de la librería *{corrplot}*



Variables categóricas y dummies

¿Cómo meto variables como sexo, color o ciudad en la regresión?

Solución: convertir a 0 y 1 (variable dummy)

Original

Persona	Sexo
Ana	Mujer
Luis	Hombre
Pía	Mujer
Juan	Hombre



Con variable dummy

Persona	Es mujer?
Ana	1
Luis	0
Pía	1
Juan	0

Regla general

Para K niveles se crean K-1 dummies.

El nivel sin dummy es la categoría base.

Para más de 2 niveles (ej. ciudades CDMX/Guadalajara/Monterrey)

Crear $K-1 = 2$ dummies.

La ciudad faltante es la *ciudad base* (*caso base*).

El R^2 no depende de cuál elijas como base.

Normalización

La "normalización" es, técnicamente, una **estandarización (z-score)**: a cada valor de la variable le resta la media y lo divide entre la desviación estándar, calculadas con los datos de entrenamiento.

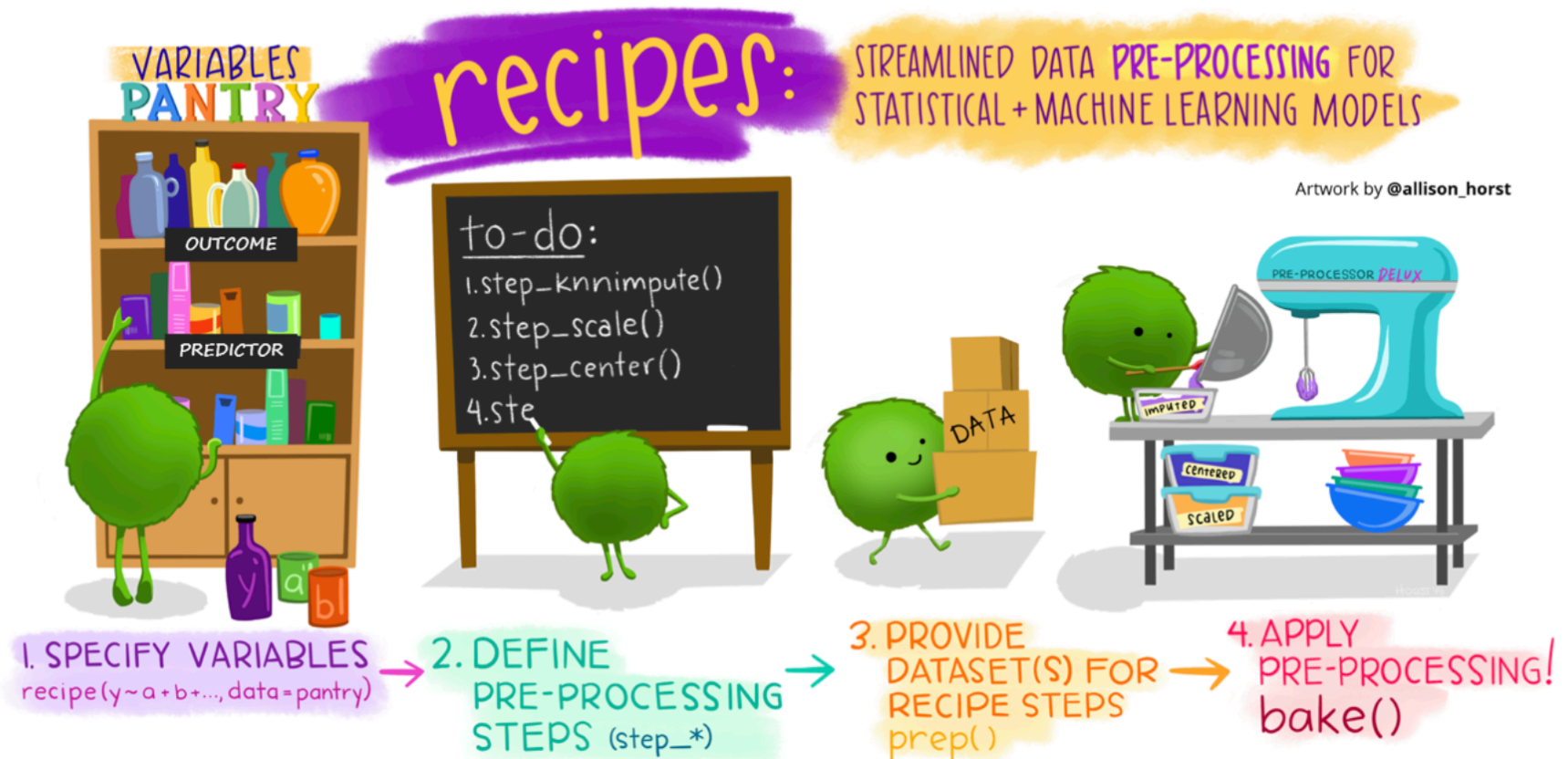
$$z_i = \frac{x_i - \bar{x}}{s_x}$$

El resultado es una variable con media 0 y desviación estándar 1. No cambia la forma de la distribución (si era sesgada, sigue siendo sesgada), solo la recentra y la reescala.

¿Cuándo importa hacerlo? En modelos sensibles a la escala: regresiones penalizadas (lasso, ridge, elastic net), KNN, SVM, k-means, PCA, redes neuronales. En árboles y bosques aleatorios es irrelevante.

Recetas

Feature engineering con el paquete {recipes}



Recetas

Supongamos el siguiente problema: quiero entrenar un modelo con una base de datos, pero mis datos tienen:

- 1) Variables con NAs.
- 2) Variables con varianza casi nula (la mayoría de los valores de la columna tienen los mismos valores)
- 3) Hay variables con colinealidad
- 4) El modelo me pide que las variables estén normalizadas
- 5) Tengo variables categóricas que son importantes (y las tengo que volver variables *dummy*)

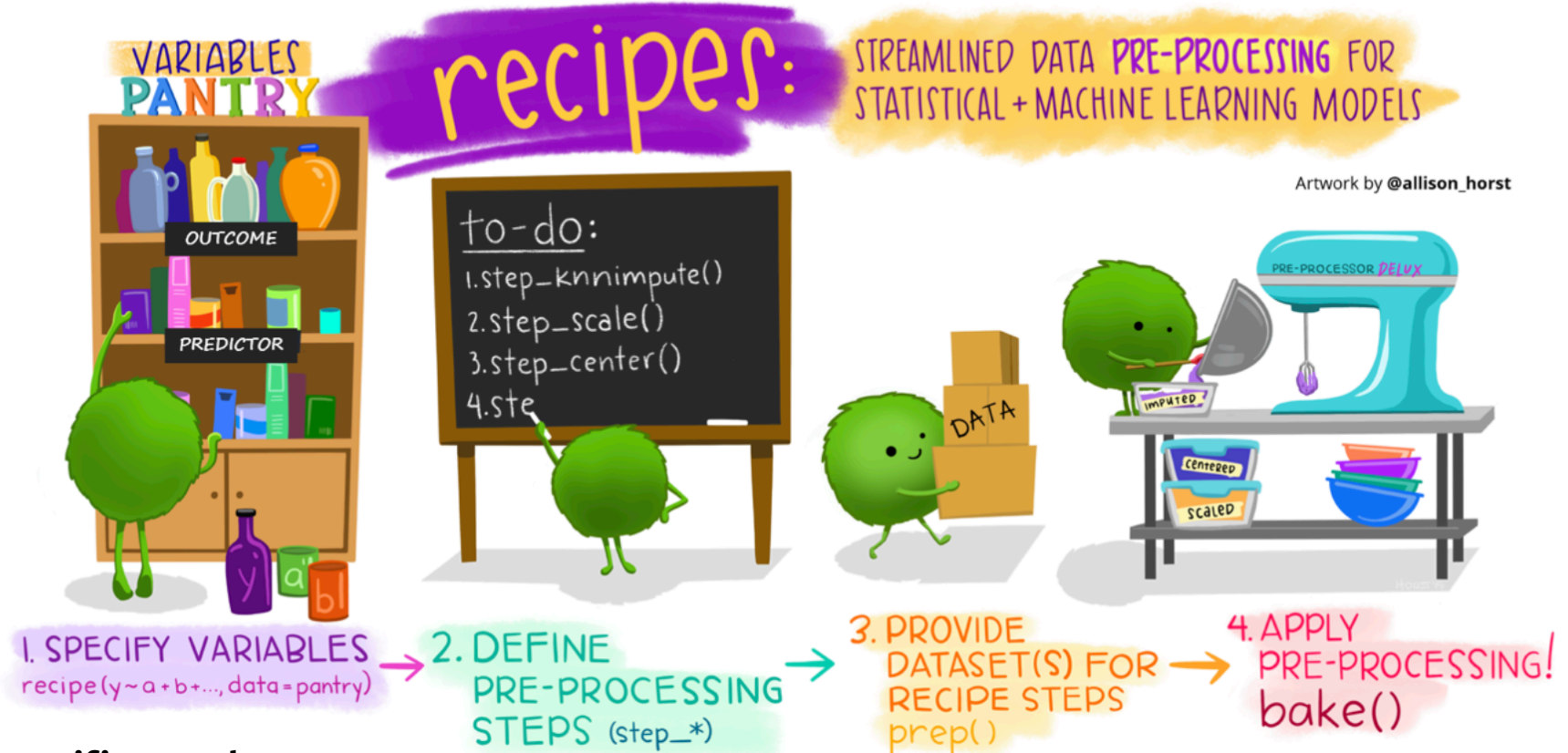
Y **además**, tengo que aplicar estas transformaciones a la **base de entrenamiento** y a la **base de prueba**.

¿Cómo aplicar estas transformaciones de forma eficiente?

Acciones de *feature engineering*

Problema	Funciones <code>step_*()</code>	Notas
Variables dummy	<code>step_dummy()</code>	Para categóricas → numéricas
Normalización / estandarización	<code>step_normalize()</code> , <code>step_center()</code> , <code>step_scale()</code> , <code>step_range()</code>	Centrar, escalar o reescalar a [0,1]
Colinealidad	<code>step_corr()</code>	Elimina predictores muy correlacionados
Datos faltantes	<code>step_impute_median()</code> , <code>step_impute_mean()</code> , <code>step_impute_mode()</code> , <code>step_impute_knn()</code> , <code>step_impute_bag()</code> , <code>step_impute_linear()</code>	De simple (media/mediana) a sofisticado (KNN, bagging)
Varianza cero o casi nula	<code>step_zv()</code> , <code>step_nzv()</code>	Útil tras crear dummies
NAs en factores	<code>step_unknown()</code>	Los convierte en nivel propio
Discretización / binning	<code>step_discretize()</code> , <code>step_cut()</code>	Continua → categórica
Variables de fecha	<code>step_date()</code> , <code>step_holiday()</code> , <code>step_time()</code>	Extrae día, mes, año, festivos, etc.
Series de tiempo	<code>step_lag()</code>	Crea rezagos
Asimetría (skewness)	<code>step_log()</code> , <code>step_sqrt()</code> , <code>step_BoxCox()</code> , <code>step_YeoJohnson()</code>	Box-Cox solo positivos; Yeo-Johnson acepta ceros y negativos

Recetas (*recipes*)



Especificamos las variables. Con la función `recipe()` generamos un objeto receta con la fórmula de regresión y la base de entrenamiento.

Definimos los pasos de preprocesamiento (Feature engineering). Vamos anidando los pasos que van a mejorar nuestra base de entrenamiento y de prueba. Estos pasos pueden ser imputar datos, eliminar variables con varianza nula, colinealidad, generar dummies, normalizar, etc.

Preparamos la receta con el dataset de prueba. Con la función `prep()` y la receta con los pasos, entrenamos la receta con los datos de entrenamiento para aplicarla a los datos de entrenamiento y validación.

Horneamos el preprocesamiento. Ya con la receta lista, usamos la función `bake()` para hornear las bases con su correspondiente *feature engineering*.

Y luego seguimos con el `fit()`

Aplicando recetas

Paso 1. Cargamos *tidymodels* y cargamos los datos



Paso 2. Dividimos la base total en base de entrenamiento y base de prueba

```
cultivos_split <- initial_split(cultivos, prop = 0.75,  
                                strata = rendimiento_ton_ha)  
  
cultivos_training <- cultivos_split %>% training()  
cultivos_test      <- cultivos_split %>% testing()  
  
print(paste("Filas en entrenamiento:", nrow(cultivos_training)))  
print(paste("Filas en prueba:",        nrow(cultivos_test)))
```

Aplicando recetas

Paso 1. Cargamos *tidymodels* y cargamos los datos



Paso 2. Dividimos la base total en base de entrenamiento y base de prueba

```
cultivos_split <- initial_split(cultivos, prop = 0.75,  
                                strata = rendimiento_ton_ha)  
  
cultivos_training <- cultivos_split %>% training()  
cultivos_test      <- cultivos_split %>% testing()  
  
print(paste("Filas en entrenamiento:", nrow(cultivos_training)))  
print(paste("Filas en prueba:",        nrow(cultivos_test)))
```

Aplicando recetas

Paso 3. Hacemos estadística descriptiva de ambas bases.



¡¡Nuevo!!

Paso 4. Feature engineering (recetas)



```
# 1. Especificamos las variables. Con la función recipe() generamos  
# un objeto receta con la fórmula de regresión y la base de entrenamiento.  
receta_cultivos <- recipe(rendimiento_ton_ha ~ .,  
                           data = cultivos_training)
```



¡¡Nuevo!!

Paso 4. Feature engineering (recetas)



Vamos concatenando los pasos del preprocesamiento del feature engineering. **Nota:** acá si importa el orden en el que se lleven a cabo los pasos.

```
# 2. Definimos los pasos de preprocesamiento (Feature engineering). Vamos  
# anidando los pasos que van a mejorar nuestra base de entrenamiento y de prueba.  
# Estos pasos pueden ser imputar datos, eliminar variables con varianza nula,  
# colinealidad, generar dummies, normalizar, etc.
```

```
receta_cultivos <- receta_cultivos %>%  
  step_impute_median(all_numeric_predictors()) %>% # Imputar con la mediana  
  step_impute_mode(all_nominal_predictors()) %>% # Imputar con la moda  
  step_nzv(all_predictors()) %>% # Quitar variables con varianza nula o chica  
  step_corr(all_numeric_predictors(), threshold = 0.85) %>% # Quitar variables colineales  
  step_normalize(all_numeric_predictors()) %>% # Normalizamos entre 0 y 1  
  step_dummy(all_nominal_predictors()) # Generamos variables dummy
```



Notas:

- * **all_predictors():** Todas las variables independientes del modelo (**X**).
- * **all_nominal_predictors():** Todas las variables independientes que son de texto o categorías.
- * **all_numeric_predictors():** Todas las variables independientes que son numéricas.
- * **all_outcomes():** Todas las variables independientes o respuesta (**y**)

¡¡Nuevo!!

Paso 4. Feature engineering (recetas)

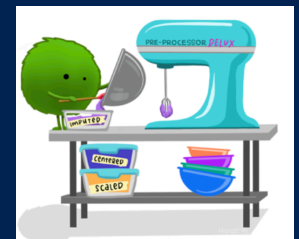
3. Preparamos la receta con el dataset de prueba. Con la función prep() y la
receta con los pasos, entrenamos la receta con los datos de entrenamiento para
aplicarla a los datos de entrenamiento y validación.

```
receta_cultivos_prep <- receta_cultivos %>%  
  prep(training = cultivos_training)
```



4. Horneamos el preprocesamiento. Ya con la receta lista, usamos la función
bake() para hornear las bases con su correspondiente feature engineering.

```
cultivos_training_prep <- receta_cultivos_prep %>%  
  bake(new_data = NULL)  
  
cultivos_test_prep <- receta_cultivos_prep %>%  
  bake(new_data = cultivos_test)
```



Nuevo paso 5. Definimos el modelo

```
modelo_lasso <- linear_reg(penalty = 0.1, mixture = 1) %>%  
  set_engine("glmnet")
```

Nuevo paso 6. Ajustamos la fórmula

```
lasso_fit <- modelo_lasso %>%  
  fit(rendimiento_ton_ha ~ ., data = cultivos_training_prep)
```

Nuevo paso 7. Predicción de valores

Nuevo paso 8. Probamos la exactitud del modelo

Repaso - Ejercicio práctico.

1. Cargue las librerías correspondientes. Utilice la función `tidymodels::tidymodels_prefer()` para evitar conflictos de funciones.
2. Cargue los datos del archivo `precio_viviendas_2.xlsx`. Explore la base.
3. Obtenga la matriz de correlación y grafíquela con `corrplot()`. ¿Hay variables **X** con colinealidad?
4. Verifique si hay NAs entre las variables. ¿Cuál es la variable con más NAs, si tienen?
5. Haga el resamdeo en base de entrenamiento y base de prueba. Que la base de entrenamiento represente el 75% de la base. Utilice `set.seed(20260512)`
6. Genere una receta que le permita: (1) imputar con la mediana los valores numéricos faltantes, (2) imputar con la moda los valores nominales faltantes, (3) elimine las variables con colinealidad (correlación mayor a 0.85), (4) elimine las variables con varianza cero y (5) convierta las variables nominales a variables dummy. La fórmula de regresión será `precio ~ .`

Repaso - Ejercicio práctico.

7. Una vez terminada la receta, aplique a las bases de entrenamiento y las bases de prueba.
8. Ya con las bases “horneadas” (sin *colinealidad* y todos esos problemas) ajuste el modelo de regresión con la función *fit()* y la fórmula correspondiente.
9. Una vez que tenga el modelo entrenado con la base de entrenamiento, prediga los valores de **y** utilizando la variable de prueba, usando la función *predict()*.
10. Una vez que tenga los datos predichos y los datos reales de la base de prueba, obtenga el rmse, el rsq y el mae (Mean Average Error).
11. Finalmente, genere una gráfica en ggplot como la de la clase pasada, donde tenga en el eje “X” los valores reales y en el eje “Y” los valores predichos. Agregue una línea diagonal con la capa *geom_abline()*.

Gracias

¿Preguntas?